



Towards Engineering LLM-Enhanced Multi-agent Systems: A Critical Examination of Roles

Tansu Zafer Asici¹(✉) , Önder Gürcan² , and Geylani Kardas¹ 

¹ Ege University, International Computer Institute, 35100 Izmir, Türkiye
91200000857@ogrenci.ege.edu.tr, geylani.kardas@ege.edu.tr

² NORCE Research AS, Center for Modeling Social Systems, Kristiansand, Norway
ongu@norce-research.no

Abstract. This paper proposes a structured approach to integrating Large Language Models (LLMs) into Multi-Agent Systems (MAS) by revisiting and extending the fundamental Agent-Oriented Software Engineering (AOSE) concept of “roles.” Traditional AOSE methodologies provide well-defined processes for modeling agents, roles, goals, and interactions, yet contemporary LLM-based MAS frameworks typically lack such systematic engineering foundations. We highlight how ad hoc development practices in LLM-enhanced MAS—often driven by prompt engineering or role-playing strategies—can lead to inconsistencies and reduced maintainability. Through a critical examination of role definition, specification, and implementation, we identify several gaps in terms of software engineering. To bridge these gaps, we propose a hybrid role-based architecture where we treat *roles as first-class entities at run-time* encapsulating both traditional AOSE design principles and LLM-driven functionalities. By laying this groundwork, we aim to foster more robust, scalable, and transparent engineering of LLM-enhanced MAS.

Keywords: Multi-Agent Systems · Large Language Models · Agent-Oriented Software Engineering · Role Abstraction · Action Execution

1 Introduction

Large Language Models (LLMs), with their advanced natural language processing capabilities, have opened new avenues for enhancing the capabilities of Multi-Agent Systems (MAS). Examples of these include improved resource coordination and decentralized collaboration [45, 85], enhanced memory and context handling with hierarchical memory models [85], advanced problem-solving [31] and simulation capabilities [26], scalability and flexibility through frameworks for rapid prototyping [15] and scalable deployment [17]. The rapid advancement of LLMs has also sparked growing interest from research and industry leveraging

agent-based solutions, accelerating the development of practical MAS applications and dedicated innovative tools. LLM-enabled MAS applications span multiple domains, including education [44], 6G communications [43], healthcare [4], financial analysis [78], agriculture [49], crime detection [70] and social simulations [32,33]. Dedicated tools and frameworks aiming to leverage the benefits of LLMs to create more efficient, autonomous, and scalable MAS, facilitating their application across various domains, are being proposed at an increasing pace [16,45,52,84].

However, the existing tools and frameworks often lack a dedicated engineering methodology, resulting in LLM-enabled MAS applications being frequently developed in an ad hoc manner, which can lead to inconsistencies and inefficiencies in their implementation and deployment. This challenge is further compounded by a significant gap in integrating LLMs within MAS, particularly in leveraging the extensive body of knowledge from Agent-Oriented Software Engineering (AOSE). AOSE provides a rich set of principles, methodologies, and tools specifically designed to address the complexities of developing agent-based systems [8,37,76]. The under-utilization of established AOSE methodologies [11,25,58,61,80] and frameworks for integrating LLMs into MAS hinders the optimization of design and implementation, ultimately affecting system quality, reliability, and maintainability. Because, by providing structured processes and clear guidelines, these methodologies help development teams systematically plan, implement, and test multi-agent software, reducing the likelihood of errors and costly rework.

Software engineering methodologies are built upon conceptual elements that serve as high-level representations (or “blueprints”) of concepts within a system. Since they do not strictly dictate the underlying details, these elements can be implemented in many ways depending on the chosen technology, programming language, system constraints, or performance needs. The AOSE literature outlines key conceptual elements such as agents, roles, goals, actions, interactions, organizational structures, communication protocols, and mental states (e.g., beliefs, desires, intentions). However, adapting them to real-world LLM-enabled MAS applications remains *undereexplored*.

Based on this observation, in this paper, as a first step towards engineering LLM-enhanced MAS, we study the role concept and its realization, as it is the standard building block for specifying agents in AOSE methodologies [1]. The contributions of this study are as follows:

- We survey and categorize current LLM-powered MAS tools and frameworks (e.g., AutoGPT, CAMEL, MetaGPT, LangChain), analyzing their development workflows and methodological limitations.
- We identify gaps between traditional AOSE methodologies and contemporary LLM-enhanced MAS implementations.
- We provide a comprehensive, structured analysis of the role concept in MAS, covering its definition, specification, and implementation from both AOSE and LLM-enhanced perspectives.

- We propose a hybrid architecture that treats roles as first-class run-time entities, encapsulating both AOSE principles and LLM functionalities.

The organization of this paper is as follows: Sect. 2 provides a brief background about engineering MAS. Section 3 describes how LLM-enabled MAS are engineered in practice and provides an evaluation in terms of AOSE. Section 4 undertakes a comparative analysis of the role concept in both AOSE and LLM contexts—across the dimensions of definition, specification, and implementation. In Sect. 5, we propose a hybrid role-based architecture that encapsulates both traditional AOSE design principles and LLM-driven functionalities. Section 6 concludes the paper by summarizing the findings, and outlines potential future work.

2 Engineering Multi-agent Systems

MAS methodologies provide structured frameworks for the analysis and design of agent-based systems, enabling agents to perform specific tasks effectively. For example, GAIA [80] offers systematic modeling for complex systems by explicitly defining agent roles, responsibilities, and interaction protocols. Roles represent specific functionalities and responsibilities assigned to agents, whereas protocols regulate interactions among agents. The MaSE [55] methodology supports the design process by analyzing systems as sets of roles and tasks. These tasks and roles are linked to agent goals and subsequently combined to define agent classes in the design phase. O-MaSE [18, 20] extends MaSE by introducing meta-models and modular components, emphasizing agent-environment interactions. This facilitates modeling of systems involving environmental processes and rules. Tropos [11] views agents as social actors structured around concepts such as goals, plans, capabilities, and beliefs. In Tropos, roles represent abstract behaviors of actors within specific contexts, while positions indicate the set of roles an actor may assume. Prometheus [58] simplifies agent design by adopting a goal- and plan-oriented approach, linking goals explicitly to functionalities. Functionalities are defined in terms of actions, messages, and data handled by agents, thereby streamlining system development. ASEME [72] takes a platform-independent, abstract, and modular approach, modeling actor behaviors and control flows by integrating actors, roles, goals, and use scenarios. In the final step, code compatible with agent development platforms such as JADE is automatically generated. The AGRE [25] model defines agents in terms of roles, groups and environments within an organizational context. This allows agents to adopt flexible and adaptive behaviors across multiple roles, facilitating the design of dynamic and modular systems (see [67] for a detailed example).

Although existing AOSE methodologies provide various tools for the structured definition of roles, responsibilities, tasks, and interactions, they lack the flexibility and creativity offered by LLMs. This constitutes a significant challenge for traditional methodologies in adapting to today’s rapidly evolving application domains.

3 Practical Engineering of LLM-Enabled MAS

This section describes the design principles and configuration approaches used by current LLM-powered MAS development tools to support practical engineering, highlighting features that enhance their overall usability.

3.1 Existing Tools and Frameworks

Current tools for developing LLM-enabled MAS can be grouped into three categories based on their primary functionality:

- Autonomous Task-Oriented Frameworks: Tools like AutoGPT¹, BabyAGI², SuperAGI³, and XAgent⁴ use iterative task loops and autonomous planning-execution mechanisms to achieve goals with minimal human intervention.
- Role-based Collaborative Frameworks: CAMEL [53], ChatDev [63], and MetaGPT [40] configure multiple agents with specialized roles, using structured interactions to collaboratively perform complex tasks, particularly in software development contexts.
- General Multi-Agent Orchestration Frameworks: AgentVerse [17], AutoGen [81], CrewAI⁵, AWS MAO⁶, OpenAI Swarm⁷, and LangChain⁸ facilitate flexible configuration and management of agent interactions, emphasizing modularity, scalability, and integration.

3.2 Common Development Workflows and Practices

Developers in practice leverage these tools to rapidly prototype and implement LLM-enabled MAS for a variety of tasks. A common workflow begins with defining the roles or objectives of each agent, often via careful prompt engineering or scripting using the framework’s API. For example, using a library like AutoGen or LangChain, a developer can instantiate agents with distinct personas (e.g., a “Planner” agent and an “Executor” agent) and then script an interaction loop between them. Typically one agent may be tasked to break down a problem and another to solve sub-problems, or one may critique and improve the other’s outputs. Similarly, with role-based systems such as ChatDev or MetaGPT, the developer provides an initial high-level task (for instance, a software feature request), and the system spawns multiple agent instances

¹ AutoGPT, <https://agpt.co/>, accessed June 26, 2025.

² BabyAGI, <https://babyagi.org/>, accessed June 26, 2025.

³ SuperAGI, <https://superagi.com/>, accessed June 26, 2025.

⁴ XAgent, <https://github.com/OpenBMB/XAgent/>, accessed June 26, 2025.

⁵ CrewAI, <https://www.crewai.com/>, accessed June 26, 2025.

⁶ AWS Multi-Agent Orchestrator, <https://awslabs.github.io/multi-agent-orchestrator/>, accessed June 26, 2025.

⁷ OpenAI Swarm, <https://github.com/openai/swarm>, accessed June 26, 2025.

⁸ LangChain, <https://python.langchain.com/>, accessed June 26, 2025.

(developer, tester, manager, etc.) that message each other to gradually refine and implement the solution. In these multi-agent conversations, each agent’s prompt template encodes its persona and scope of work, which guides the agent’s behavior throughout the dialogue. Developers often iterate on these prompts to steer the agents toward productive interactions (e.g. ensuring the “tester” agent knows how to systematically find faults in the “developer” agent’s code output).

Another prevalent development pattern is the use of autonomous task loops, epitomized by AutoGPT and BabyAGI. In this pattern, once a user supplies an initial goal, the agent system itself iteratively generates sub-tasks, executes them, evaluates results, and adjusts the plan or creates new tasks as needed. Developers employ such patterns to offload not just single-step queries to LLMs, but entire project workflows. For instance, a user might ask AutoGPT to “research and write a report on market trends”, upon which the system will autonomously break the job into smaller tasks (e.g. data gathering, analysis, drafting) and cycle through them until completion. In practice, frameworks like SuperAGI or AWS’s MAO provide higher-level interfaces for running these autonomous agents, allowing developers to configure resources (for example, a vector database for long-term memory) and to monitor the agent’s progress through a dashboard.

Building LLM-enabled MAS today is an iterative and experimental process. Developers rely heavily on observing agent behaviors through logs or real-time monitors and then refining the system. If the MAS fails to converge to a good solution, one might adjust the prompts, add a new agent (for example, an agent whose sole role is to critique solutions or to summarize the discussion so far), or tweak the interaction protocol (for instance, enforcing that agents communicate in a structured format or brief bullet points to reduce ambiguity). In summary, current best practices involve defining clear agent roles, endowing agents with the necessary tools or external knowledge sources, simulating their interactions in a controlled environment, and iteratively improving their prompts and logic. Over time, repeated patterns (such as a planner-executor pair, or a questioner-responder-reviewer trio) are becoming templates that developers can reuse across projects, gradually forming an evolving playbook for how to effectively combine multiple LLM agents for different classes of problems.

3.3 Methodologies and Guidelines in Current Tools

Given the relative infancy of LLM-enabled MAS, most of the existing tools do not prescribe a rigorous software engineering methodology for design or implementation, nor do they explicitly reference classical AOSE frameworks (e.g., Gaia, Tropos) from the MAS literature. Development is often guided by example use cases and the built-in templates provided by each tool, rather than by a formal process model. For instance, AutoGPT and BabyAGI emerged as experimental prototypes shared via open-source repositories; users typically follow the provided README or community-written guides to adapt them to new tasks, engaging in a lot of trial-and-error. The emphasis is on achieving functional goals (e.g., “*let the agent autonomously handle my email workflow*”) and tweaking the agent’s prompts or code as needed when issues arise, rather than on adherence

to a standardized development lifecycle. In essence, the methodology is *ad hoc* - developers iteratively refine the system until it performs the desired behavior, which is feasible given the high-level nature of LLM prompts but lacks the guarantees of traditional engineering approaches.

Some frameworks do offer informal guidelines aligned with their design philosophies. For example, MetaGPT incorporates the concept of Standardized Operating Procedures (SOPs) from human teamwork as part of its prompting strategy. This provides a semi-structured template for how agents should collaborate on a complex task (e.g., a defined sequence of phases such as requirement analysis → design → implementation → testing in a software project). A developer using MetaGPT is encouraged to follow this template when setting up agent roles and conversation order, which is a form of methodology albeit specific to the software-development domain. Similarly, ChatDev’s enforced structure of a virtual software team implicitly serves as a development guideline: it suggests that to solve a problem (say, build a new software feature), one should instantiate a team of agents with complementary roles and have them communicate in a logical order reflecting a typical software engineering process. These tool-specific conventions (often inspired by real-world workflows) provide starting points for developers, but they are not generalizable frameworks one could apply to any MAS project in a systematic way. Notably, we find little to no evidence that these new platforms build on established AOSE methodologies – for instance, their documentation and papers do not mention using Gaia’s role models or Tropos’s goal diagrams to design agent societies. The emphasis is more on empirical effectiveness (does a given configuration of agents solve the task?) rather than a priori design correctness. In practice, the lack of formal methodology means that much of the development knowledge is tacit, residing in the experience of the developers or shared through blog posts and forums rather than encoded in the frameworks themselves.

3.4 Incorporation of Software Engineering Principles

Current LLM-enabled MAS frameworks implicitly support software engineering principles like modularity and reusability by dividing complex problems into specialized agents with clearly defined roles. Tools such as LangChain emphasize role specialization, which simplifies the development, testing, and potential reuse of agent components across projects. Role specialization aligns well with the principle of single responsibility, enhancing overall system maintainability.

However, significant challenges persist. The behavior of agents, defined largely through prompts, lacks transparency and formal analyzability, complicating maintainability—minor prompt changes can unpredictably influence entire systems. Frameworks like AutoGen offer visual debugging to mitigate this, but robust, systematic methods remain scarce. Additionally, runtime-based traceability provided by logging agent interactions doesn’t easily map back to original requirements, limiting systematic verifiability and role clarity.

Scalability also poses issues. While selective message routing and orchestrators, such as those employed by OpenAI Swarm and AWS MAO, improve

manageability of agent interactions by efficiently handling multiple agent roles, empirical evidence of their effectiveness in large-scale scenarios remains limited.

Overall, while current frameworks demonstrate foundational support for modular, reusable agent roles, the absence of formal methodologies, traceability mechanisms, and comprehensive scalability strategies highlights critical areas needing development to fully integrate classical software engineering principles into practical MAS engineering.

4 A Critical Examination of Roles

AOSE has long provided solid analytical and design foundations for MAS. AOSE methodologies encompass fundamental concepts such as agents, roles, goals, actions, interactions, organizational structures, and communication protocols. Among these concepts, the *role* is a key concept as it is used to define responsibilities and capabilities of agents. In many AOSE methodologies, a common method is handling agent roles and then aggregate them to form complete agents [1]. Consequently, as a first step towards bringing AOSE methodologies and the practical LLM-enabled MAS development together, we start by critically examining the role concept which is a fundamental conceptual element in AOSE.

In software engineering, conceptual elements are typically characterized by three facets: their definition, specification, and implementation. Accordingly, we will examine, in order, the role concept's definition (Sect. 4.1), specification (Sect. 4.2), and implementation (Sect. 4.3).

4.1 Role Definition

In AOSE, a role is an abstract definition that encapsulates a set of *responsibilities*, *behaviors*, and *interaction protocols* expected of an agent within a MAS. By decoupling the expected behaviors from the agent's concrete implementation, the role concept promotes modularity, reusability, and flexibility, allowing agents to adopt or change roles dynamically as system requirements evolve. Roles specify what actions an agent should perform, how it should interact with other agents, and what obligations or constraints it must observe in a given organizational, social, or physical context (also called environment). Environments define the context, resources, norms, and possible interactions available to agents [79]. They shape and constrain how roles are interpreted and executed, enabling situated reasoning and behavior adaptation.

Different methodologies approach the concept of role in various ways. GAIA [80] emphasizes that roles encompass responsibilities, protocols, and permissions to perform specific functions. MaSE [55] structures roles through use case scenarios and sequence diagrams during system specification. O-MaSE [18, 20] extends MaSE to include environmental interactions, deepening the modeling of roles. Tropos [11] defines a role as an abstract representation of the behavior of a social actor in a specific context, while Cabri et al. [13] highlight the interactional aspects of this approach. Prometheus considers a role as an element that reflects

system functionality and defines the actions performed by agents. ASEME [74] presents dynamic role models derived from system use case scenarios, modeling the influence of roles on agent-internal behavior through control flow. Finally, AGRE [25] defines a role as a functional position within a group, emphasizing that multiple agents can share the same role as an abstraction.

In AOSE, roles are typically an *analysis* and/or *design-time* concept rather than a *run-time* concept. In some cases (e.g. GAIA), the role concept is only used during the analysis phase and does not exist in the design. Over the years, one of the significant problems faced by the MAS community was the transition of the concept of a role from an abstract design to a concrete implementation within an agent framework, mainly because roles often did not exist at run-time, making effective mapping challenging [9].

In the context of an LLM, a *role* can be defined as a predetermined behavioral profile or identity assigned to the model (such as “an experienced medical advisor” or “a technical support expert”). This role guides the model’s responses, language style, information prioritization, and context awareness to effectively perform specific tasks or interactions. Here, the role concept is a *design-time* concept, interpreted and executed by the LLM at *run-time*. However, an LLM role does not inherently encapsulate interaction protocols—structured sequences of messages and actions that govern multi-agent communication [56]. Instead, it primarily defines the responsibilities and behaviors needed to produce appropriate, task-specific responses. While LLMs do not natively support interaction protocols, such protocols can be implemented and managed externally using recently proposed dedicated interoperability protocols [24]. For example, Model Context Protocol (MCP)⁹ servers can be used to coordinate and mediate communication between LLM-based agents, ensuring adherence to protocol logic, turn-taking, and state management¹⁰ [68]. Moreover, even though an LLM role can interact with environments mediated by tools and frameworks, the relationship between roles and environments can not explicitly be defined or integrated within the LLM itself [54, 56]. This poses problems in multi-agent settings¹¹.

At *design-time*, the developer defines what role the model will assume and how it should behave, while at *run-time*, the LLM actively applies and adapts these guidelines dynamically during user interactions. However, at *run-time*, the LLM processes these guidelines as instructions or contextual cues rather than maintaining explicit “roles” and “actions” in the agent-oriented sense (with goals, behaviors, or organizational structures). Essentially, the concept of roles and actions is supplied and interpreted by users and developers through prompts and conversation structure, rather than by the LLM’s own inherent representation. While LLMs can appear to perform roles and actions (e.g., adding two numbers), this “execution” is still just a text-based transformation (generating a

⁹ MCP, <https://modelcontextprotocol.io/>, accessed on June 26, 2025.

¹⁰ See, as an example, Coral Server of the Coral Protocol, <https://github.com/Coral-Protocol/coral-server>, accessed on June 26, 2025.

¹¹ Cognition, <https://cognition.ai/blog/dont-build-multi-agents>, accessed on June 26, 2025.

numeric answer) rather than an explicit representation of action, roles, or goals as understood in AOSE. It is effectively a pattern matching or textual reasoning process, not an agent-based execution driven by internally modeled objectives.

4.2 Role Specification

The specification of a role provides a detailed, often formal, description of the actions it can perform and how it can interact with other roles [35,47]. The behavior is often described in terms of permissions, responsibilities, activities, and interactions [5]. This detailed breakdown is used to guide the implementation of the role. In the following, we give an ordered list of methods for specifying roles, arranged by increasing levels of formality and precision—from the most informal, conceptual descriptions to the most rigorous, mathematically precise and verifiable specifications:

1. **Natural Language Description:** The use of natural language in role definition offers both accessibility and flexibility. In this approach, users define a role by explicitly and comprehensively describing its tasks, responsibilities, interaction patterns, and objectives using natural language expressions.
2. **Graphical Modeling Languages** (e.g., UML¹², SysML¹³, AUML¹⁴): These provide visual diagrams to depict roles, their relationships, and interactions. They strike a balance between clarity and formality, making them well suited for conceptual modeling and communication among stakeholders [6,30,71].
3. **Business Process Modeling Languages** (e.g., BPMN¹⁵): Designed for illustrating business workflows, BPMN specifies roles within processes clearly. It is more structured than free-form graphics yet still primarily serves as a communication tool rather than a formal specification [57].
4. **Architectural Description Languages (ADLs)** (e.g., ACME [27], AADL [3]): ADLs focus on system architecture, detailing how roles (often as components or connectors) integrate into and interact within an overall system. They add structure by specifying interfaces and inter-component relationships [23,60].
5. **Programming Code:** Implementing role specifications in programming languages makes them executable. While this approach is concrete and precise in terms of behavior, it often intertwines specification with implementation details rather than remaining purely abstract [66].
6. **Domain-Specific Languages (DSLs):** DSLs are custom-tailored to particular domains. They capture role details and constraints that are specific to the domain context, offering both precision and relevance while potentially varying in formal rigor [21].
7. **Behavioral Modeling Languages** (e.g., Statecharts [36], Petri nets [62]): These languages emphasize the dynamic aspects of roles—modeling state transitions, interactions over time, and concurrency. They provide a formal way to describe how roles behave under various conditions [14,73].

¹² OMG UML, <https://www.omg.org/spec/UML>, accessed on June 26, 2025.

¹³ OMG SysML, <https://www.omg-sysml.org/>, accessed on June 26, 2025.

¹⁴ AUML, <https://auml.org/auml/>, accessed on June 26, 2025.

¹⁵ BPMN, <https://www.omg.org/bpmn/>, accessed on June 26, 2025.

8. Ontology and Semantic Web Languages (e.g., OWL¹⁶, RDF¹⁷): By defining roles within a network of semantically related concepts, these languages offer a formal framework that supports logical inference and interoperability. They are especially useful when roles need to be integrated into larger knowledge representation systems [34, 41].
9. Logic Programming Languages (e.g., Prolog [29]): Using a declarative, rule-based approach, logic programming languages allow you to specify roles in terms of logical constraints and relationships. This method supports formal reasoning and can automatically infer properties of roles [39].
10. Formal Specification Languages (e.g., Z [75], Alloy [42], TLA⁺ [50], VDM [46], B [2]): These are the most mathematically rigorous methods available. They provide precise, unambiguous specifications that can be used for formal proofs and verification, ensuring that role properties and interactions meet strict correctness criteria [28, 59, 64].

These various methods for role specification comprehensively outline the theoretical framework and expectations of a role within a system. These methods not only determine which actions a role will perform and how it will interact with other roles but also provide crucial insights into how these abstract concepts can be transformed into concrete actions during the implementation process.

In the context of LLMs and LLM-enabled agents, role specification relies solely on natural language descriptions. These textual inputs are interpreted and refined through a detailed prompt engineering process, enabling LLMs to capture the user’s intent. This process ensures that the generated responses align with the desired expertise or narrative style, thereby improving overall performance. In essence, natural language serves as a vital bridge between human intent and model behavior—the clarity and scope of the language used in defining a role directly shape the quality and relevance of the model’s outputs. However, to achieve less error-prone role specifications, it is preferable to use the more precise methods described above. These specifications can then be *translated* into natural language descriptions when interacting with LLMs.

Besides, role specialization and reusability are partially feasible within LLM-based systems, particularly when supported by structured prompt engineering and modular design frameworks. By defining roles through detailed persona descriptions—encompassing aspects such as background, expertise, tone, and communication style—LLMs can consistently emulate specific behaviors across various contexts. This approach not only enhances the model’s ability to maintain a coherent persona but also facilitates the reuse of these role definitions across different tasks and applications. However, challenges remain in ensuring that specialized roles do not lead to overfitting or reduced generalization capabilities. Techniques such as Role Prompting Guided Domain Adaptation (REGA) [77] address this by preserving the general capabilities of LLMs while allowing for effective domain-specific adaptations. REGA employs strategies like

¹⁶ OWL, <https://www.w3.org/TR/owl-ref/>, accessed on June 26, 2025.

¹⁷ RDF, <https://www.w3.org/TR/rdf11-concepts/>, accessed on June 26, 2025.

self-distillation and role integration to mitigate issues like catastrophic forgetting and inter-domain confusion. Having said that, while frameworks and methodologies are emerging to bring more structure to LLM role definitions, LLMs still lack the formal structure of object-oriented programming (OOP), making it challenging to ensure consistency and reusability across different contexts, and they do not yet match the rigor and predictability offered by OOP paradigms.

4.3 Role Implementation

A role defines a set of actions that an agent can perform to achieve its objectives within a MAS. These actions are guided by the structural arrangements associated with the role, which may include interacting with the environment, gathering information, making decisions, and communicating with other agents. By adopting roles, agents can dynamically respond to changing conditions and collaborate effectively on complex tasks. The specific actions associated with a role can be implemented in the following ways.



Fig. 1. Actions Directly Called within Code.

Actions Directly Called Within Code. Roles are implemented using programming languages and are invoked directly by agents within their code. As shown in Fig. 1, an action is first added to the relevant role. Then, Agent requests to perform the role. For an Agent to perform its Role, it has to execute various actions. After execution, actions produce outcomes, which are returned to the agent by the role. This method is more common in AOSE methodologies such as GAIA [83], Prometheus [58], MaSE [55] and is based on the implementation of role definitions directly as methods or plans in the agent code.

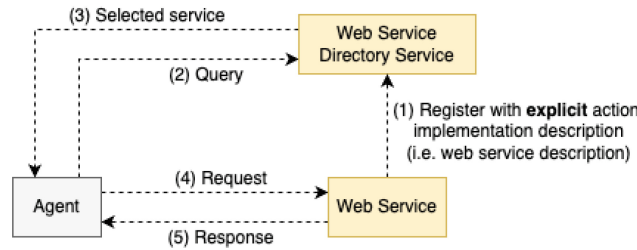


Fig. 2. Actions Called as Semantic Web Services.

Actions Called as Semantic Web Services. Agents dynamically discover and invoke actions via services, aligning with modern microservices architectures. In Fig. 2, firstly the action as *Web Service* registers its capabilities to Directory Service. Then, at any time, Agent sends a request to find the appropriate service. Once selected, *Web Service* is invoked, performs the required operations, and returns the output information in (5) [12].

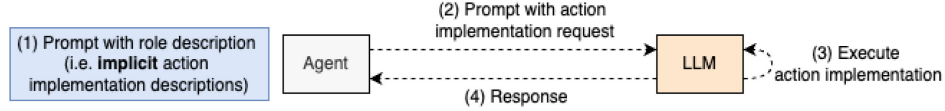


Fig. 3. Actions Triggered for LLM to Perform Its Role.

Actions Triggered for LLM to Perform Its Role. An LLM autonomously performs tasks by assuming a specific role, enabling artificial intelligence to exhibit autonomous behavior. In the first step of Fig. 3, Agent initiates a two-stage prompt process before forwarding it to LLM. In (2), Role supplies its description, and in (3), it sends a prompt containing the action request. In (4), LLM executes the required action-whether generating a response, handling a query, or performing another designated task and returns the output or action outcome back to Agent [17, 53, 63].

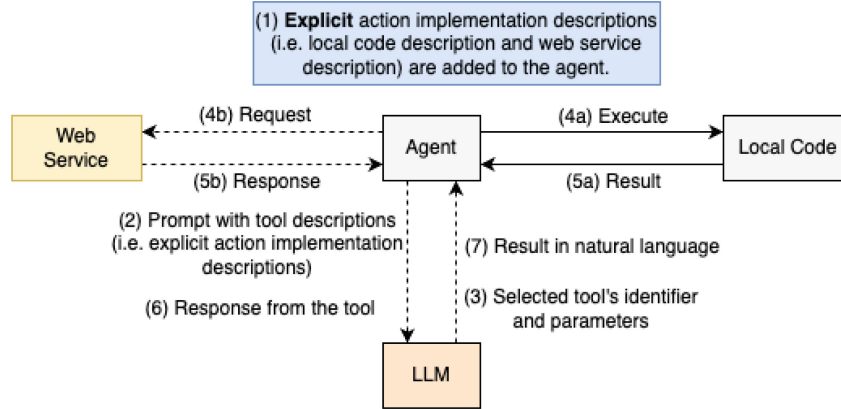


Fig. 4. Actions Triggered by LLM-controlled Tools.

Actions Triggered by LLM-Controlled Tools. Agents use natural language queries or commands to trigger actions through LLMs, leveraging advancements in AI and natural language processing. As shown in Fig. 4, in (1), a specific

action definition is explicitly added to the Agent. Then, in (2), the Agent sends a prompt to the LLM, including descriptions of the defined tools or actions. In (3), the LLM selects the most suitable tool from the available options. The chosen action is then either executed by the Agent within *Local Code* in (4a) or performed by sending a request to the *Web Service* in (4b). In (5a), the result from the *Local Code* is returned to the Agent. In (5b), the response from the *Web Service* is returned to the Agent. In (6), the Agent forwards these results to the LLM. Finally, in (7), the LLM translates these results into natural language and returns them to the Agent.

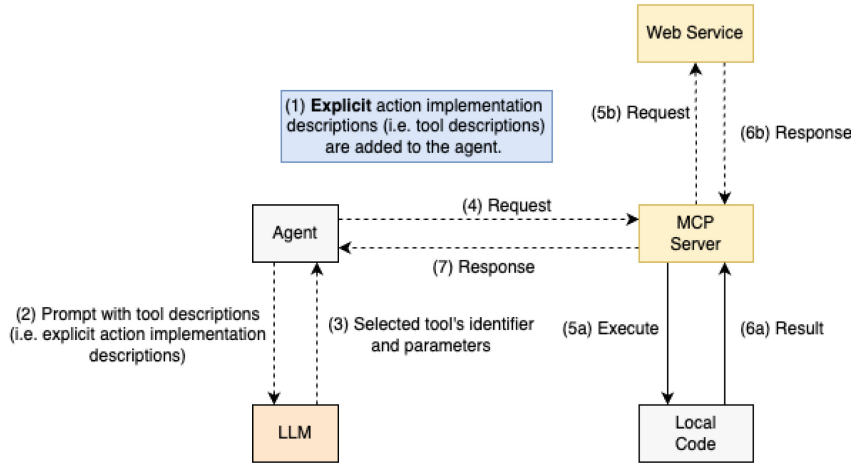


Fig. 5. Actions Triggered by LLM-controlled MCP.

Actions Triggered by LLM-Controlled MCP. MCP is a robust and open protocol specifically designed to facilitate seamless integration of data sources and external tools into LLM-enhanced MAS. MCP operates on a client-server model, systematically managing resources (e.g., documents, databases), tools (e.g., API calls, file operations), and standardized prompt templates required by LLMs. As shown in Fig. 5, MCP Server provides these resources and tools, while LLM acting as MCP Client establishes connections, enabling Agent to efficiently discover and utilize available resources.

In LLM-supported MAS, the selection of role implementation methods must be carefully evaluated based on various criteria, including system performance, security, flexibility, and maintenance ease. Directly invoked actions within code provide developers with full control, enabling efficient debugging, performance optimization, and security. While this method ensures minimal latency and high computational efficiency, its static nature limits flexibility, leading to issues such as code redundancy, maintenance complexity, and scalability challenges. On the other hand, actions invoked as semantic web services enhance dynamic adaptability and reusability through runtime updates and service extensions. While

centralized maintenance facilitates cross-platform integration, dependency on external service providers introduces risks such as API changes, service outages, security vulnerabilities, and network latency [48]. Actions triggered by LLM-controlled tools simplify complex operations via natural language interactions, providing a user-friendly environment and supporting rapid prototyping [69]. However, this approach comes with the risks of unpredictable LLM behavior [22], potential errors [82], and high resource consumption [19]. Additionally, actions triggered by an LLM assuming a role offer high adaptability, creativity, and continuous learning capabilities. The ability to assign roles using natural language further enhances accessibility. However, this method also presents challenges such as a lack of transparency [38], security and privacy risks [65], and significant computational demands. Finally, the LLM-controlled Model Context Protocol (MCP) provides a standardized layer for resource and tool discovery, thereby offering flexibility, but it also raises protocol complexity and initial deployment costs. Consequently, architectural design must consider the performance, security, scalability, and maintainability dimensions to determine the most appropriate implementation method for each role.

In this section, we have shown that roles are not necessarily explicitly represented an implementation of a concept. In AOSE, roles are often used as abstract concepts during *design-time* and typically lack explicit representation at *run-time*. However, this absence hinders the clear mapping between agents' behaviors and their corresponding roles, thereby limiting the system's dynamic adaptability. As highlighted in frameworks such as ROPE [7] and JaCaMo [10], this limitation underscores the need for explicit run-time role modeling. Lhaksmana et al. [51] further stress the critical advantages of defining roles explicitly at run-time, particularly in self-organizing MAS. We thus argue that, in LLM-supported MAS, roles should be treated as *first-class entities at run-time* to effectively manage diverse application domains and support adaptability. Such an approach enhances agents' ability to adapt dynamically, increases run-time flexibility, and promotes overall system sustainability. This shift toward explicit run-time role representation is embodied in the hybrid role-based architecture presented in Sect. 5.

5 Proposed Role-Based Architecture

Based on the aforementioned observations, this section lays the foundation for an initial architecture for LLM-enhanced MAS engineering. Our goal is to establish a comprehensive framework that integrates traditional AOSE concepts (e.g., role, role implementation) with LLM-based dynamic capabilities. Due to the requirements of hybrid usage and encapsulation, we propose maintaining role definitions explicitly within agents. By encapsulation, we mean bundling data and the actions that operate on that data in a role into a single unit. This way, we can protect the role's internal state from unintended interference or misuse. Moreover, encapsulation promotes modularity and maintainability, making it easier to modify and debug the role while ensuring data integrity.

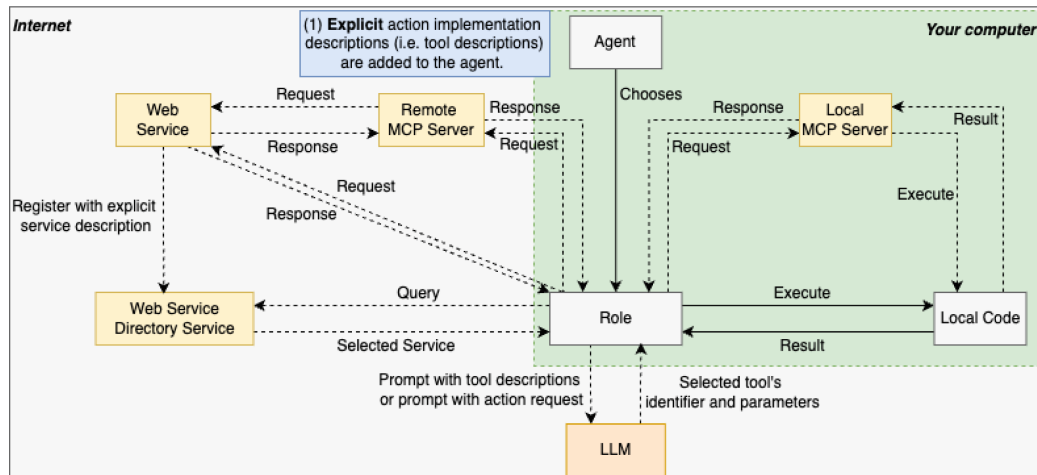


Fig. 6. Proposed Role-based Architecture.

Role definitions can encompass one or multiple role implementation approaches outlined in Sect. 4.3 (Fig. 6). This means that a role may involve actions that are executed locally, actions accessed via web services, tasks performed by locally implemented LLM tools, or operations carried out directly by LLMs.

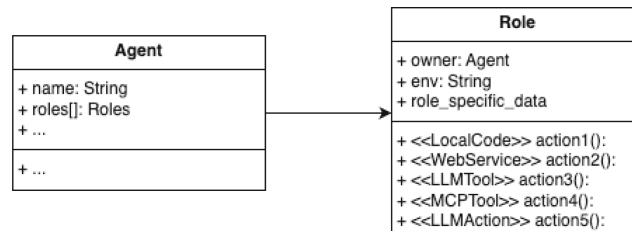


Fig. 7. Proposed Role Concept in UML Class Diagram.

Consequently, we design the LLM-enabled role concept using the UML graphical modeling language since it is more precise compared to natural language. Figure 7 shows a simple UML class model of an agent with roles. Each Role belongs to one Agent (its owner), has a string indicating the environment where the role is played, and may include various role specific data. A Role can have five types of “actions” with different stereotypes—indicating, for example, LocalCode, WebService, LLMTTool, MCPTTool, or LLMAction—that can be invoked when an agent is acting in that particular role.

When an agent intends to determine or guide its actions within a specific role using outputs generated by an LLM, it employs a dynamically generated prompt. This prompt is constructed by selecting one of the two prompt templates depicted in Fig. 8. Figure 8(a) illustrates a scenario in which the

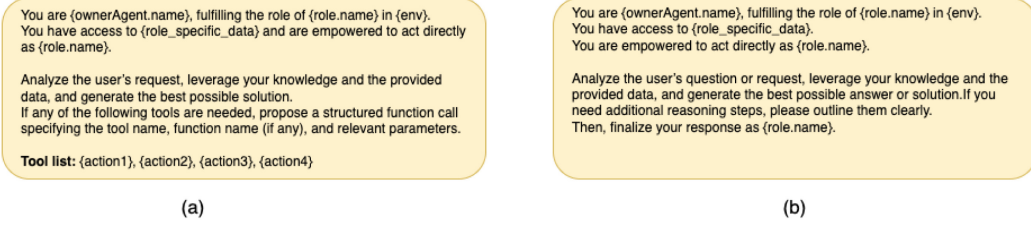


Fig. 8. (a) A prompt template for a role configured to use tools when necessary. (b) A prompt template for a role that generates a response directly through the LLM.

agent performs an action by utilizing external tools based on recommendations provided by the LLM, whereas Fig. 8(b) represents a scenario in which the agent directly utilizes the textual output from the LLM as its response. Placeholders in these prompts—such as `ownerAgent.name`, `role.name`, `env`, and `role_specific_data`—are dynamically replaced at runtime with concrete values derived from the respective role instance and its owning agent. Consequently, the LLM obtains explicit contextual information about the agent’s identity, the role being enacted, the operational environment, and the available capabilities. Thus, the agent interprets the output generated by the LLM and either executes appropriate actions through designated tools or directly incorporates the LLM’s response into its communication processes.

6 Conclusion

In this study, we shed light on the fact that existing LLM-based tools and frameworks do not sufficiently leverage AOSE’s extensive body of knowledge. While LLM-based frameworks and tools have made impressive strides in enabling powerful and flexible agent behavior, they often do so through ad hoc design approaches that compromise maintainability, reusability, and scalability.

To address this issue, we conducted a focused study on the concept of roles, a foundational element in AOSE, and examined how it is defined, specified, and implemented in both traditional AOSE and LLM-based systems. Our analysis revealed key differences and limitations in current LLM-enabled role modeling, particularly the absence of formal structure and runtime support.

However, adapting the concept of “role” in LLM-assisted MAS development and transforming it into concrete actions brings both new opportunities and challenges. Hybrid solutions that combine the solid principles of traditional AOSE methodologies with the innovative dynamism of LLMs can be effective in overcoming these challenges. Based on this observation, we proposed a hybrid role-based architecture that encapsulates both traditional AOSE design principles and LLM-driven functionalities. This initial architecture is able to express roles that have various actions types while encapsulating them.

By bridging the conceptual foundations of AOSE with the dynamic potential of LLMs, our work lays the groundwork for a more principled engineering

methodology for LLM-enhanced MAS. Future work will focus on refining this architecture and validating its effectiveness through practical case studies involving real-world MAS applications.

References

1. Abdalla, R., Mishra, A.: Agent-oriented software engineering methodologies: analysis and future directions. *Complexity* **2021**(1), 1629419 (2021)
2. Abrial, J.R., Hoare, A., Chapron, P.: *The B-Book* (1996)
3. Aerospace, S.: SAE architecture analysis and design language (AADL) annex volume 1: Annex a: Graphical AADL notation. Annex C: AADL Meta-Model and Interchange Formats, Annex D: Language Compliance and Application Program Interface Annex E: Error Model Annex, AS5506/1 (2011)
4. AlMutairi, M., AlKulaib, L., Aktas, M., Alsalamah, S., Lu, C.T.: Synthetic Arabic medical dialogues using advanced multi-agent LLM techniques. In: *Proceedings of The Second Arabic Natural Language Processing Conference*, pp. 11–26 (2024)
5. Bakar, M., Ghoul, S.: A methodology for AUML role modeling, pp. 74–81 (2011). <https://doi.org/10.1109/ISIICT.2011.6149600>
6. Bauer, B., Odell, J.: UML 2.0 and agents: how to build agent-based systems with the new UML standard. *Eng. Appl. Artif. Intell.* **18**(2), 141–157 (2005)
7. Becht, M., Gurzki, T., Klarmann, J., Muscholl, M.: Rope: role oriented programming environment for multiagent systems. In: *Proceedings Fourth IFCIS International Conference on Cooperative Information Systems. CoopIS 99* (Cat. No. PR00384), pp. 325–333. IEEE (1999)
8. Bergenti, F., Gleizes, M.P., Zambonelli, F.: *Methodologies and Software Engineering for Agent Systems: The Agent-oriented Software Engineering Handbook*, vol. 11. Springer Science & Business Media (2006). <https://doi.org/10.1007/b116049>
9. Beydoun, G., et al.: FAML: a generic metamodel for mas development. *IEEE Trans. Softw. Eng.* **35**(6), 841–863 (2009)
10. Boissier, O., Bordini, R.H., Hübner, J.F., Ricci, A., Santi, A.: Multi-agent oriented programming with JACAMO. *Sci. Comput. Program.* **78**(6), 747–761 (2013)
11. Bresciani, P., Perini, A., Giorgini, P., Giunchiglia, F., Mylopoulos, J.: TROPOS: an agent-oriented software development methodology. *Auton. Agent. Multi-Agent Syst.* **8**, 203–236 (2004)
12. Cabral, L., Domingue, J., Motta, E., Payne, T., Hakimpour, F.: Approaches to semantic web services: an overview and comparisons. In: Bussler, C.J., Davies, J., Fensel, D., Studer, R. (eds.) *ESWS 2004. LNCS*, vol. 3053, pp. 225–239. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-25956-5_16
13. Cabri, G., Leonardi, L., Puviani, M.: Service-oriented agent methodologies. In: *16th IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE 2007)*, pp. 24–29. IEEE (2007)
14. Celaya, J.R., Desrochers, A.A., Graves, R.J.: Modeling and analysis of multi-agent systems using petri nets. In: *2007 IEEE International Conference on Systems, Man and Cybernetics*, pp. 1439–1444. IEEE (2007)
15. Chandrasekaran, M.: Enhancing efficiency and flexibility of rapid prototyping for scalable multimodal intelligent agents. In: *2024 Artificial Intelligence for Business (AIB)*, pp. 66–71. IEEE (2024)

16. Chen, G., et al.: AutoaGents: a framework for automatic agent generation, pp. 22–30 (2024)
17. Chen, W., et al.: AgentVerse: facilitating multi-agent collaboration and exploring emergent behaviors (2024)
18. Dam, K.H., Winikoff, M.: Comparing agent-oriented methodologies. In: Giorgini, P., Henderson-Sellers, B., Winikoff, M. (eds.) AOIS -2003. LNCS (LNAI), vol. 3030, pp. 78–93. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-25943-5_6
19. Dang, Y., He, Y., Xu, M., Ye, K.: Resource management for GPT-based model deployed on clouds: challenges, solutions, and future directions. LNCS (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 15252, pp. 95–105 (2025). https://doi.org/10.1007/978-981-96-1528-5_7
20. DeLoach, S.A., Valenzuela, J.L.: An agent-environment interaction model. In: Padgham, L., Zambonelli, F. (eds.) AOSE 2006. LNCS, vol. 4405, pp. 1–18. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-70945-9_1
21. Demirkol, S., Challenger, M., Getir, S., Kosar, T., Kardas, G., Mernik, M.: Sea_l: a domain-specific language for semantic web enabled multi-agent systems. In: 2012 Federated Conference on Computer Science and Information Systems (FedCSIS), pp. 1373–1380. IEEE (2012)
22. Deng, Y., Zhang, W., Pan, S.J., Bing, L.: Multilingual jailbreak challenges in large language models (2024)
23. Dissaux, P., et al.: The smart project: multi-agent scheduling simulation of real-time architectures. In: Embedded Real-Time Software System (2014)
24. Ehtesham, A., Singh, A., Gupta, G.K., Kumar, S.: A survey of agent interoperability protocols: model context protocol (MCP), agent communication protocol (ACP), agent-to-agent protocol (A2A), and agent network protocol (ANP). arXiv preprint [arXiv:2505.02279](https://arxiv.org/abs/2505.02279) (2025)
25. Ferber, J., Michel, F., Baez, J.: AGRE: integrating environments with organizations. In: Weyns, D., Van Dyke Parunak, H., Michel, F. (eds.) E4MAS 2004. LNCS (LNAI), vol. 3374, pp. 48–56. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-32259-7_2
26. Gao, C., et al.: Large language models empowered agent-based modeling and simulation: a survey and perspectives. *Humanit. Soc. Sci. Commun.* **11**(1) (2024). <https://doi.org/10.1057/s41599-024-03611-3>
27. Garlan, D., Monroe, R., Wile, D.: ACME: An Architecture Description Interchange Language, White Paper. Pittsburgh PA, submitted for publication, Computer Science Department, Carnegie Mellon University (1997)
28. Ghafarollahi, A., Buehler, M.J.: Automating alloy design and discovery with physics-aware multimodal multiagent AI. *Proc. Natl. Acad. Sci.* **122**(4), e2414074122 (2025). <https://doi.org/10.1073/pnas.2414074122>
29. Giannesini, F., Kanoui, H., Pasero, R., Van Caneghem, M.: Prolog. Addison-Wesley Longman Publishing Co., Inc (1986)
30. Guedes, G.T.A., Vicari, R.M.: Applying AUML and UML 2 in the multi-agent systems project. In: Heuser, C.A., Pernul, G. (eds.) ER 2009. LNCS, vol. 5833, pp. 106–115. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-04947-7_14
31. Guo, T., et al.: Large language model based multi-agents: a survey of progress and challenges, pp. 8048–8057 (2024)

32. Gurcan, O.: LLM-augmented agent-based modelling for social simulations: challenges and opportunities. In: HHAI 2024: Hybrid Human AI Systems for the Social Good, *Frontiers in Artificial Intelligence and Applications*, vol. 386, pp. 134–144. IOS Press (2024). <https://doi.org/10.3233/FAIA240190>
33. Gurcan, O., Falck, V., Rousseau, M.G., Lima, L.L.: Towards an LLM-powered social digital twinning platform. In: *Proceedings of The 23rd International Conference on Practical applications of Agents and Multi-Agent Systems (PAAMS 2025)* (2025)
34. Haarslev, V., Möller, R.: RACER: an owl reasoning agent for the semantic web. In: *Proceedings of the International Workshop on Applications, Products and Services of Web-based Support Systems*, in conjunction with the, pp. 91–95 (2003)
35. Hameurlain, N., Sibertin-Blanc, C.: Specification of role-based interactions components in multi-agent systems. *LNCS (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* **3390**, 180–197 (2005). https://doi.org/10.1007/978-3-540-31846-0_11
36. Harel, D.: StateCharts: a visual formalism for complex systems. *Sci. Comput. Program.* **8**(3), 231–274 (1987)
37. Henderson-Sellers, B., Giorgini, P.: *Agent-oriented methodologies*. IGI Global (2005)
38. Hepenstal, S., Zhang, L., Wong, B.L.W.: The impact of system transparency on analytical reasoning (2023). <https://doi.org/10.1145/3544549.3585786>
39. Hjelmblom, M.: Deontic action-logic multi-agent systems in Prolog. *Högskolan i Gävle* (2008)
40. Hong, S., et al.: MetaGPT: meta programming for a multi-agent collaborative framework. In: *The Twelfth International Conference on Learning Representations* (2023)
41. Hui, K., Chalmers, S., Gray, P.M.D., Preece, A.D.: Experience in using RDF in agent-mediated knowledge architectures. In: van Elst, L., Dignum, V., Abecker, A. (eds.) *AMKM 2003. LNCS (LNAI)*, vol. 2926, pp. 177–192. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-24612-1_12
42. Jackson, D.: Alloy: a lightweight object modelling notation. *ACM Trans. Softw. Eng. Methodol. (TOSEM)* **11**(2), 256–290 (2002)
43. Jiang, F., et al.: Large language model enhanced multi-agent systems for 6G communications. *IEEE Wirel. Commun.* **31**(6), 48–55 (2024). <https://doi.org/10.1109/MWC.016.2300600>
44. Jiang, Y.H., Liu, T.Y., Zhuang, X., Hu, H., Li, R., Jia, R.: Enhancing educational practices with multi-agent systems: a review. In: *Enhancing Educational Practices: Strategies for Assessing and Improving Learning Outcomes*, pp. 47–65 (2024)
45. Jin, A., Ye, Y., Lee, B., Qiao, Y.: DecoAgent: large language model empowered decentralized autonomous collaboration agents based on smart contracts. *IEEE Access* **12**, 155234–155245 (2024). <https://doi.org/10.1109/ACCESS.2024.3481641>
46. Jones, C.B.: *Systematic Software Development Using VDM*, vol. 2. Prentice Hall Englewood Cliffs (1990)
47. Koning, J.L., Hernández, I.R.: Limitations in AUML’S roles specification. *IFIP Adv. Inf. Commun. Technol.* **163**, 79–82 (2005). https://doi.org/10.1007/0-387-23152-8_10
48. Kumar, D., et al.: Security challenges in an increasingly tangled web, pp. 677–684 (2017). <https://doi.org/10.1145/3038912.3052686>
49. Kuska, M.T., Wahabzada, M., Paulus, S.: AI for crop production – where can large language models (LLMs) provide substantial value? *Comput. Electron. Agric.* **221**, 108924 (2024). <https://doi.org/10.1016/j.compag.2024.108924>

50. Lamport, L.: Specifying Systems, vol. 388. Addison-Wesley Boston (2002)
51. Lhaksmana, K.M., Murakami, Y., Ishida, T.: Role-based modeling for designing agent behavior in self-organizing multi-agent systems. *Int. J. Software Eng. Knowl. Eng.* **28**(01), 79–96 (2018)
52. Li, C., et al.: ModelScope-agent: building your customizable agent system with open-source large language models, pp. 566–578 (2023)
53. Li, G., Hammoud, H., Itani, H., Khizbullin, D., Ghanem, B.: Camel: communicative agents for “mind” exploration of large language model society. *Adv. Neural. Inf. Process. Syst.* **36**, 51991–52008 (2023)
54. Li, X.: A review of prominent paradigms for LLM-based agents: Tool use (including rag), planning, and feedback learning, vol. Part F206484-1, p. 9760–9779 (2025)
55. Loach, S.A.D., Wood, M.: Developing multiagent systems with agentTool. In: Castelfranchi, C., Lespérance, Y. (eds.) ATAL 2000. LNCS (LNAI), vol. 1986, pp. 46–60. Springer, Heidelberg (2001). https://doi.org/10.1007/3-540-44631-1_4
56. Malfa, E.L., et al.: Large language models miss the multi-agent mark (2025)
57. Onggo, B.S., Karpát, O.: Agent-based conceptual model representation using BPMN. In: Proceedings of the Winter Simulation Conference (WSC), 2011, pp. 671–682. IEEE (2011)
58. Padgham, L., Winikoff, M.: Prometheus: a practical agent-oriented methodology. In: Agent-Oriented Methodologies, pp. 107–135. IGI Global (2005)
59. Paiva, P.Y.A., Saotome, O., Brandauer, C.: Specification and verification of a multi-agent coordination protocol with TLA+. In: 2018 VIII Brazilian Symposium on Computing Systems Engineering (SBESC), pp. 207–212. IEEE (2018)
60. Park, S., Sugumaran, V.: Designing multi-agent systems: a framework and application. *Expert Syst. Appl.* **28**(2), 259–271 (2005)
61. Pavón, J., Gómez-Sanz, J.J., Fuentes, R.: The ingenias methodology and tools. In: Agent-oriented methodologies, pp. 236–276. IGI Global (2005)
62. Petri, C.: Kommunikation mit automaten: Phd thesis/institut für instrumentelle mathematik.-bonn, 1962 (1962)
63. Qian, C., et al.: ChatDev: communicative agents for software development. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 15174–15186 (2024)
64. Ramzan, M., Ali, A., Akram, S., Qayyum, Z.U.: Formal specification of multi-agent environment using VDM-SL. In: 2011 6th Int. Conference on Computer Sciences and Convergence Information Technology (ICCIT), pp. 150–154. IEEE (2011)
65. Rathod, V., Nabavirazavi, S., Zad, S., Iyengar, S.S.: Privacy and security challenges in large language models, pp. 746–752 (2025). <https://doi.org/10.1109/CCWC62904.2025.10903912>
66. Ross, R., Collier, R., O’Hare, G.M.P.: AF-APL – bridging principles and practice in agent oriented languages. In: Bordini, R.H., Dastani, M., Dix, J., El Fallah Seghrouchni, A. (eds.) ProMAS 2004. LNCS (LNAI), vol. 3346, pp. 66–88. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-32260-3_4
67. Roussille, H., Gürçan, Ö., Michel, F.: AGR4BS: a generic multi-agent organizational model for blockchain systems. *Big Data Cognit. Comput.* **6**(1) (2022). <https://doi.org/10.3390/bdcc6010001>
68. Sarkar, A., Sarkar, S.: Survey of LLM agent communication with MCP: a software design pattern centric review. *arXiv preprint arXiv:2506.05364* (2025)
69. Sathe, G., Choudhary, V., Bhagat, D.: Comprehensive review on large language models (LLMs). **2**, 3097–3102 (2024)

70. Sathya, J., Fernandez, F.M.H.: An optimizing crime detection in social media platforms using multiagent ontology-based approach, pp. 956–966 (2023). <https://doi.org/10.1109/ICOSEC58147.2023.10276325>
71. Sha, Z., Le, Q., Panchal, J.H.: Using SYSML for conceptual representation of agent-based models. In: Int. Design Engineering Technical Conferences and Computers and Information in Engineering Conference, vol. 54792, pp. 39–50 (2011)
72. Spanoudakis, N., Moraitis, P.: Using ASEME methodology for model-driven agent systems development. In: International Workshop on Agent-Oriented Software Engineering, pp. 106–127. Springer (2010). https://doi.org/10.1007/978-3-642-22636-6_7
73. Spanoudakis, N.I.: Engineering multi-agent systems with StateCharts: theory and practice. *SN Comput. Sci.* **2**(4), 317 (2021)
74. Spanoudakis, N.I., Moraitis, P.: The ASEME methodology. *Int. J. Agent-Oriented Softw. Eng.* **7**(2), 79–107 (2022)
75. Spivey, J.: *The Z Notation*. Prentice-Hall (1992)
76. Sturm, A., Shehory, O.: Agent-oriented software engineering: revisiting the state of the art. In: *Agent-Oriented Software Engineering: Reflections on Architectures, Methodologies, Languages, and Frameworks*, pp. 13–26 (2014)
77. Wang, R., et al.: Role prompting guided domain adaptation with general capability preserve for large language models. In: Duh, K., Gomez, H., Bethard, S. (eds.) *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 2243–2255. Association for Computational Linguistics, Mexico City, Mexico (2024). <https://doi.org/10.18653/v1/2024.findings-naacl.145>
78. Wawer, M., Chudziak, J.A., Niewiadomska-Szynkiewicz, E.: Large language models and the Elliott wave principle: a multi-agent deep learning approach to big data analysis in financial markets. *Appl. Sci. (Switzerland)* **14**(24) (2024). <https://doi.org/10.3390/app142411897>
79. Weyns, D., Omicini, A., Odell, J.: Environment as a first class abstraction in multiagent systems. *Auton. Agent. Multi-Agent Syst.* **14**, 5–30 (2007)
80. Wooldridge, M., Jennings, N.R., Kinny, D.: The GAIA methodology for agent-oriented analysis and design. *Auton. Agent. Multi-Agent Syst.* **3**, 285–312 (2000)
81. Wu, Q., et al.: AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In: *First Conference on Language Modeling* (2024)
82. Ye, J., et al.: ToolsWord: unveiling safety issues of large language models in tool learning across three stages, vol. 1, p. 2181–2211 (2024). <https://doi.org/10.18653/v1/2024.acl-long.119>
83. Zambonelli, F., Jennings, N.R., Wooldridge, M.: Developing multiagent systems: the GAIA methodology. *ACM Trans. Softw. Eng. Methodol. (TOSEM)* **12**(3), 317–370 (2003)
84. Zhang, C., et al.: PROAGENT: building proactive cooperative agents with large language models, vol. 38, p. 17591–17599 (2024). <https://doi.org/10.1609/aaai.v38i16.29710>
85. Zou, H., Li, R., Sun, T., Wang, F., Li, T., Liu, K.: Cooperative scheduling and hierarchical memory model for multi-agent systems. In: *2024 IEEE International Symposium on Product Compliance Engineering-Asia (ISPCE-ASIA)*, pp. 1–6. IEEE (2024)